ADR-S-026: Named Compositions, Five-Level Stack, and Intent Vector Envelope

Series: S (specification-level — applies to all implementations) **Status**: Accepted **Date**: 2026-03-08 **Scope**: Formal system extension — macro layer, gap.intent output type, intent vector as orchestration envelope

Review record: - Proposed: Claude Code `20260308T110000` (five-level stack + named compositions) and `20260308T120000` (intent vector unification) - Gemini review: `20260308T130000` — directionally aligned, raised macro registry requirement - Codex review: `20260308T183553` — accepted 11:00 stack as primary architectural step; corrected tuple, source_kind distinctions, convergence vocabulary; rated 12:00 as promising but premature as replacement - Ratified: 2026-03-08 (consensus; all Codex corrections incorporated)

Context

The formal system defines four primitives and one operation. The eight named functors in `HIGHER_ORDER_FUNCTORS.md` give these primitives their operational vocabulary: BROADCAST, FOLD, BUILD, CONSENSUS, REVIEW, DISCOVERY, RATIFY, EVOLVE.

Two gaps remained after that document was written:

Gap 1 — Macro layer: Common workflow patterns (plan a work program, run a proof-of-concept, discover a dataset's schema) cannot be expressed without repeating the same functor sequences. The eight primitive functors are composable but the composition has no stable name, no reuse mechanism, and no dispatch target for the gap evaluator.

Gap 2 — IntentEngine output type: When the gap evaluator detects a delta and raises an intent, it currently produces free-text findings. Free text requires interpretation before the result can be executed. An interpreter introduces ambiguity the system has no contract to resolve. The evaluator's output type should be as typed as any other evaluator output.

Gap 3 — Orchestration envelope: The system tracks feature vectors (trajectories through the graph) but has no formal object for the unit of work dispatched to resolve each gap or advance each trajectory. Product vectors, discovery vectors, design vectors, and feature vectors are all conceptually the same thing — intent to produce an asset using a named composition — but the vocabulary has been informal and inconsistent.

This ADR addresses all three gaps as a coherent extension of the formal system.

Decision

1. The Five-Level Stack

The formal system is extended to recognise five compositional levels:

```
Level 1: Primitives         {Graph, Iterate, Evaluators,
Spec+Context}
          ↓ compose into
Level 2: Named Functors     {BROADCAST, FOLD, BUILD, CONSENSUS,
REVIEW, DISCOVERY, RATIFY, EVOLVE}
          ↓ compose into
Level 3: Named Compositions  {PLAN, POC, SCHEMA_DISCOVERY,
DATA_DISCOVERY, ...}
          ↓ compose into
Level 4: Project Expressions {composition.yml — the authored artifact}
          ↓ compiles to
Level 5: Graph Topology     {graph_topology.yml + edge_params/ — the
execution model}
```

Level 1 is unchanged — the four primitives are the axioms.

Level 2 is the eight named functors defined in `HIGHER_ORDER_FUNCTORS.md`. These are building blocks, not domain patterns.

Level 3 (new) is the macro layer. Named compositions are reusable workflow patterns defined entirely in terms of Level 2 functors. They: - Are named and versioned in a composition library - Accept typed parameter bindings - Emit a typed stable asset when they converge - May nest other named compositions - Are the vocabulary of the IntentEngine's `intent_raised` output

Level 4 (new) is the project-level composition — the sequence of named compositions and functors that defines a project type (standard SDLC, data pipeline, governance change). This is what the user authors.

Level 5 is the compiled execution model — what the system actually runs. Level 4 compiles to Level 5. The graph_topology.yml is a Level 5 artifact.

The five levels do not replace the existing graph model. The existing graph topology (Level 5) is unchanged. Named compositions (Level 3) become the authoring vocabulary that compiles down to Level 5 structure.

---

2. Named Compositions (Level 3)

A named composition is a macro: a functor sequence with named parameters, a typed output, and a stable identity in the composition library.

2.1 Formal definition

```
NamedComposition(
    name: identifier,
```

```
    parameters: {name: type, ...},
    body: functor_sequence,
    output: AssetType,
    profile: {standard | poc | spike | ...}
) → output
```

The `body` is a sequence of Level 2 functor applications. Named compositions may be nested inside other named compositions.

2.2 Initial library

PLAN — work-program planning for planning-heavy asset transitions:

```
PLAN(source_asset, unit_type, criteria) ≡
  BROADCAST(source_asset, decompose_fn: unit_type)      # 1→N:
decompose into units
  ∘ iterate(each_unit, evaluators: criteria)             # evaluate
each independently
  ∘ FOLD(units, merge_fn: dep_dag_builder)               # N→1: dep
DAG + build order
  → work_order {units, dep_dag, build_order, ranked_units,
deferred_units}
```

PLAN terminates at `work_order`. It does not include the REVIEW/CONSENSUS gate or the subsequent construction step. The full planning chain is:

```
asset_T → PLAN → work_order → REVIEW/CONSENSUS? → CONSTRUCT →
asset_T+1
```

PLAN is not a universal intermediary between every asset transition. Hotfix skips it. Post-code lifecycle stages do not obviously fit the decompose/evaluate/order/rank pattern. PLAN is a reusable composition for planning-heavy transitions — this is its scope.

The current `feature_decomposition` and `design_recommendations` nodes in the SDLC graph are candidates for PLAN, but their equivalence has not been formally proved. Do not collapse `graph_topology.yml` nodes until the common output schema, shared Markov criteria, and asset-specific invariants (e.g., ADR-S-013 coverage requirements on `feature_decomposition`) have been mapped. The correct migration path: introduce a shared `plan.yml` edge parameter template behind the existing nodes first; unify the topology only after the schema is stable.

POC — proof-of-concept: risk-bounded exploration that converges on learning, not delivery:

```
POC(intent, risk_areas, time_box) ≡
  PLAN(intent, unit_type: risk_area, criteria: risk_reduction_value)
  ∘ BUILD(spike_code ↔ spike_tests)
  ∘ DISCOVERY(findings, convergence: question_answered, time_box:
time_box)
  → poc_report {question, findings, risk_disposition: resolved |
remains | new_risk_identified}
```

The spike code is an explicit byproduct, not a deliverable. The `risk_disposition` field determines what happens next: - `resolved` → the parent vector proceeds - `new_risk_identified` → a new POC spawns - `remains` → the parent vector is blocked pending a human decision

SCHEMA_DISCOVERY — infer a schema document from a raw dataset:

```
SCHEMA_DISCOVERY(dataset, notebook_env, evaluation_criteria) ≡
  BROADCAST(dataset, sample_fn: stratified_sample)
  ∘ iterate(each_sample, evaluators: [structure_detect, type_infer,
null_rate, cardinality])
  ∘ FOLD(sample_schemas, merge_fn: schema_unifier)
  ∘ REVIEW(unified_schema, evaluator: F_H, criteria: completeness +
fitness_for_use)
  → schema_document {fields, types, nullability, cardinality,
examples, open_questions}
```

Every field in the `schema_document` traces back to which sample, which BROADCAST path, which iterate() invocation.

DATA_DISCOVERY — map an unknown data domain across multiple sources:

```
DATA_DISCOVERY(data_sources[], question, time_box) ≡
  BROADCAST(data_sources, explore_fn: data_source)
  ∘ SCHEMA_DISCOVERY(each_source, ...)        # nested composition
  ∘ FOLD(source_schemas, merge_fn: relationship_mapper)
  ∘ DISCOVERY(relationship_map, convergence: question_answered)
  → data_landscape {sources, schemas, relationships, gaps,
open_questions}
```

SCHEMA_DISCOVERY nests inside DATA_DISCOVERY. Named compositions compose.

2.3 Composition library governance

Change type	Gate
New named composition (library-level, shared across projects)	CONSENSUS (architecture committee, per ADR-S-025)
New named composition (project-local)	REVIEW (F_H, per edge_params)
Parameter change to existing composition (non-breaking)	REVIEW
Breaking change to existing composition	CONSENSUS
Deprecation	CONSENSUS

Compositions are versioned. A composition expression names both the macro and the version: `PLAN@v1`, `SCHEMA_DISCOVERY@v1`. If version is omitted, the current library-pinned version applies.

3. gap.intent Emits Typed Composition Expressions

The IntentEngine's typed_output categories (§VIII of the Bootloader) are: - `reflex.log` — zero ambiguity, fire-and-forget - `specEventLog` — bounded ambiguity, deferred iteration - `escalate` — persistent ambiguity, judgment required

Currently, `specEventLog` and `escalate` produce free-text findings. This ADR replaces free text with **typed composition expressions**:

```
{
  "event_type": "intent_raised",
  "intent_id": "INT-DATA-003",
  "composition": {
    "macro": "SCHEMA_DISCOVERY",
    "version": "v1",
    "bindings": {
      "dataset": "data/raw/transactions.parquet",
      "notebook": "jupyter://dev-env",
      "review": "F_H"
    }
  },
  "vector_type": "discovery",
  "gap_type": "missing_schema",
  "triggered_by": "data_pipeline→data_contract edge — no
        schema_document found"
}
```

The gap evaluator maps `gap_type → named composition` using a dispatch table that lives in the composition library alongside the compositions themselves.

Gap type → named composition dispatch table (initial):

Gap type	Named composition	Notes
`missing_schema`	SCHEMA_DISCOVERY	dataset exists, no schema
`missing_requirements`	PLAN(intent, capability, user_value)	intent exists, no requirements
`missing_design`	PLAN(requirements, feature, mvp_value)	requirements exist, no design
`unknown_risk`	POC(intent, risk_areas)	risk not yet assessed
`unknown_domain`	DATA_DISCOVERY	data landscape not mapped
`spec_drift`	EVOLVE(spec, delta)	spec and implementation

Gap type	Named composition	Notes
		diverged
`missing_consensus`	CONSENSUS(proposal, roster, quorum)	change needs ratification

Execution contract caveat: A typed composition expression is more precise than free text — it names the macro, version, and parameter bindings. But it is not yet zero-interpretation. Execution still requires: - A macro registry that resolves the named composition - Binding validation rules - A compilation rule from macro to graph fragment or operator expansion - A scope policy (library-global vs project-local)

These form the execution contract for named compositions. Implementations must define this contract before claiming that `intent_raised` events are directly executable. The typed expression removes interpretation from the gap content; the registry removes interpretation from the macro name; together they approach, but do not yet reach, zero-interpretation execution.

---

4. Intent Vector as Orchestration Envelope

4.1 The unification

Every artifact the system produces — a feature, a discovery, a design, a POC report — is the convergence result of an intent to produce an asset using a named composition. This is the same construct at different positions in the causal chain with different composition expressions bound. The vocabulary (feature vector, discovery vector, design vector, product vector) names the same underlying object.

An **intent vector** is the formal envelope for a unit of dispatched work:

```
intent_vector:
  id: {REQ-F-* | INT-* | GAP-* | SCHEMA-* | ...}
  source_kind: abiogenesis | gap_observation | parent_spawn
  trigger_event: {event reference | null}
  parent_vector: {id | null}
  resolution_level: intent | requirements | design | code | deployment
        | telemetry
  target_asset_type: {AssetType}
  composition_expression:
    macro: PLAN | POC | SCHEMA_DISCOVERY | DATA_DISCOVERY | BUILD |
        DISCOVERY | ...
    version: {v1 | ...}
    bindings: {parameter: value, ...}
  profile: standard | full | poc | spike | hotfix | minimal
  status: pending | iterating | converged | blocked | time_box_expired
  produced_asset_ref: {asset path or identifier | null while
        iterating}
  disposition: {null while iterating | converged | blocked_accepted |
        blocked_deferred | abandoned}
  vector_type: feature | discovery | spike | poc | hotfix   # kept for
        profile routing
```

Formal tuple:

```
V = (id, source_kind, trigger_event, parent_vector,
     resolution_level, target_asset_type,
     composition_expression, profile, status,
     produced_asset_ref, disposition)
```

4.2 The three sources of intent

Every intent vector originates from exactly one of three sources:

`source_kind`	Description	Example
`abiogenesis`	Human creates the first intent, bootstrapping the constraint surface	"Build an authentication system" at session start
`gap_observation`	The IntentEngine detects a delta between observed state and spec	`missing_schema`, `tolerance_breach`, `spec_drift`
`parent_spawn`	A vector intentionally decomposes work and spawns a child	Requirements vector discovers an unknown risk, spawns a POC vector

`parent_spawn` is not reducible to `gap_observation`. Some child vectors are created because the parent intentionally decomposes work — planned fan-out, specialisation, parallel exploration — not because it detected a deficit. Preserving `parent_spawn` as a distinct source_kind retains this causal distinction.

The three sources do not collapse to two. All three are first-class.

`trigger_event` is the concrete event or condition that caused this vector to be created. It is `null` only for `abiogenesis` vectors. Separating `source_kind` (category), `trigger_event` (concrete cause), and `parent_vector` (lineage link) gives the traceability model the causal richness it needs.

4.3 The causal chain

Every intent vector has a parent, except one: the abiogenesis. The chain from abiogenesis to any produced artifact is unbroken:

```
abiogenesis (human intent → spec + constraint surface)
  ↓ spawns
intent_vector[0]  source: abiogenesis, resolution: intent
  composition: PLAN(intent, capability, user_value)
  → work_order[requirements]
  ↓ REVIEW → CONSTRUCT →
intent_vector[1]  source: parent_spawn, parent: [0], resolution:
```

```
requirements
  composition: PLAN(requirements, feature, mvp_value)
  → work_order[design]
  ↓ REVIEW → CONSTRUCT →
intent_vector[2]  source: parent_spawn, parent: [1], resolution:
design
  composition: PLAN(design, module, arch_stability)
  → work_order[code]
  ↓ REVIEW → CONSTRUCT →
intent_vector[3]  source: parent_spawn, parent: [2], resolution: code
  composition: BUILD(code ↔ unit_tests)
  → stable_code_asset
  ↓ telemetry observes running system
intent_vector[4]  source: gap_observation, trigger:
latency_breach(p99=450ms, threshold=200ms)
  composition: POC(latency_issue, risk_areas: [db_query, cache_miss])
  → new branch of the causal chain
```

4.4 Resolution level

`resolution_level` is where in the graph the intent vector is currently positioned. The same underlying intent refines progressively:

```
intent          → "user needs to log in securely"
requirements    → REQ-F-AUTH-001..004 (WHAT the system must do)
design          → AuthModule + TokenService + SessionStore (HOW
architecturally)
code            → auth.py + test_auth.py (HOW concretely)
deployment      → auth-service:v1.2 (HOW operationally)
telemetry       → login_latency, auth_failure_rate (HOW observably)
```

Each level is a refinement of the same intent. Each refinement is itself an intent vector — an intent to produce the next asset in the chain.

4.5 Produced asset and disposition

`produced_asset_ref` is `null` while the vector is iterating. On convergence it records the stable asset produced (file path, artifact identifier, or asset address). This field is what makes the traceability claim complete: without it, the vector explains why work was spawned but not what resulted.

`disposition` records the terminal state in human-readable form: - `null` — vector is active (pending, iterating, or time-boxed) - `converged` — produced_asset_ref is populated, all evaluators passed - `blocked_accepted` — explicitly accepted as a known limitation - `blocked_deferred` — deferred to a future iteration or feature - `abandoned` — intentionally dropped with documented rationale

4.6 vector_type retention

`vector_type` (feature | discovery | spike | poc | hotfix) is retained for profile routing — it selects which edges are in scope and which evaluators apply. It is not an ontological category; it is a profile selector. The unified construct is the intent vector; the type is

operational annotation.

---

5. Project Convergence Vocabulary

The project is the set of all intent vectors descending from the abiogenesis, plus all intent vectors generated by gap observations during the project's lifetime. The event log is the record of all iterate() invocations across all vectors.

Three distinct terminal states replace the single "all converged or blocked" definition:

State	Definition
`project_quiescent`	No vector is actively iterating. The project is at rest but may not be done.
`project_converged`	All **required** intent vectors are `converged`. The project has produced all planned artifacts.
`project_bounded`	No vector is actively iterating; remaining `blocked` vectors have each been explicitly `accepted`, `deferred`, or `abandoned` with documented disposition.

`project_bounded` is not `project_converged`. A project can be declared bounded over unresolved planned work only when every blocked vector has been explicitly dispositioned. Documented-but-unresolved is a legitimate terminal state for a bounded project; it is not success.

A running project always has at least one vector `iterating`.

Homeostasis: the system is homeostatic when the gap evaluator's outputs (new intent vectors) are bounded — the rate of gap-generated vectors decreases as the system approaches the spec. At steady state, new gap vectors are generated only by environmental changes (ecosystem drift, new requirements) — not by structural defects in the current implementation.

Consequences

What this changes

1. **HIGHER_ORDER_FUNCTORS.md** gains a formal §6 defining named compositions (PLAN, POC, SCHEMA_DISCOVERY, DATA_DISCOVERY) and the gap type → composition dispatch table. This ADR is the authority; the document is updated to reference it.

2. **IntentEngine output** (§VIII of the Bootloader) is updated: `specEventLog` and `escalate` emit typed composition expressions, not free text. The Bootloader's §VIII is updated with this constraint.

3. **Feature vector schema** is extended to the full intent vector tuple. All existing feature vector fields are preserved; new fields (`source_kind`, `trigger_event`, `target_asset_type`, `produced_asset_ref`, `disposition`) are added. Existing vectors without these fields are treated as `source_kind: abiogenesis` or `source_kind: parent_spawn` as appropriate.

4. **Project status vocabulary** gains `quiescent`, `converged`, `bounded` as first-class states. `gen-status` and STATUS.md must distinguish these three.

5. **Composition library** is a new governed artefact. Its initial location, format, and CONSENSUS governance process are implementation-defined (each tenant decides its registry). The dispatch table (gap_type → macro) lives alongside the compositions.

What this does NOT change

- The four primitives and one operation — unchanged
- The eight named functors in HIGHER_ORDER_FUNCTORS.md — unchanged
- The current graph_topology.yml — not touched by this ADR. Feature decomposition and design_recommendations nodes remain explicit until the common PLAN schema is proved by shared edge parameter templates.
- The ADR-S-025 CONSENSUS semantics — unchanged
- The existing feature vector trajectory model — extended, not replaced

Constraints on implementations

1. An implementation MUST define a composition registry before claiming intent_raised events are executable without interpretation.
2. An implementation MUST validate composition expressions against the registry at dispatch time.
3. Named compositions MUST be defined purely in terms of Level 2 functors (or other named compositions). A named composition that introduces a new execution primitive violates the level separation.
4. Project status reporting MUST distinguish `quiescent`, `converged`, and `bounded`. Reporting `converged` when blocked vectors exist with undocumented disposition is a reporting invariant violation.
5. Intent vectors MUST record `produced_asset_ref` on convergence. An intent vector that claims `status: converged` with `produced_asset_ref: null` is invalid.

Open Questions

OQ-1: Composition compilation contract

Who owns the rule that maps a named composition (Level 3) to a graph fragment or edge sequence (Level 5)? Options: the composition library itself (self-describing), each tenant's compiler, or a shared compilation spec. Resolution: deferred to first tenant that implements a registry. The contract must be formally specified before that tenant ships.

OQ-2: CONSTRUCT naming

What is the canonical name for the functor that takes an approved work_order and produces the next asset? For code it is BUILD. For requirements and design it is `iterate()` with the work_order as primary context. Is there a single named composition (`CONSTRUCT`) or is it parameterised iterate()? This ADR does not resolve it. Implementations may name it locally; a shared name (CONSTRUCT@v1) will be proposed once the pattern is confirmed across two or more edges.

OQ-3: scope policy

When a project-local named composition has the same name as a library-level composition, which takes precedence? Resolution: project-local always shadows library-level (analogous to local variable scoping). The registry must make shadowing explicit and auditable.

Rationale Summary

The five-level stack is the primary architectural contribution: it gives the system a stable macro layer (Level 3) without pretending the existing graph/runtime disappears. Named compositions are macros; macros are defined in terms of functors; functors are defined in terms of primitives. The levels are clean and the dependencies flow in one direction.

The gap.intent typing closes the IntentEngine loop algebraically — the gap evaluator's output is now in the same type system as the rest of the system. The execution contract caveat is honest: the typed expression improves precision but does not eliminate the need for a registry.

The intent vector unification is a promising orchestration object, not yet the universal replacement vocabulary. Codex's assessment is accepted: the 11:00 five-level stack is the durable architectural step; the intent vector tuple is extended to carry the minimum fields required for the traceability claim to be honest; the asset/runtime vocabulary is not erased. The three source_kinds are kept distinct. The project convergence vocabulary is precise.

References

- `specification/core/HIGHER_ORDER_FUNCTORS.md` — eight named functors (Level 2)
- `specification/core/AI_SDLC_ASSET_GRAPH_MODEL.md` — four primitives, §5.3 lower-bound constraints, §VIII IntentEngine
- `specification/adrs/ADR-S-025-consensus-functor.md` — CONSENSUS evaluator (Level 2)
- `specification/adrs/ADR-S-013-completeness-visibility.md` — feature_decomposition invariants (preserved)
- `.ai-workspace/comments/claude/20260308T110000_STRATEGY_Intent-Vectors-As-Composition-Expressions.md` — proposal

- `.ai-workspace/comments/claude/20260308T120000_STRATEGY_Intent-Vector-Unification.md` — proposal
- `.ai-workspace/comments/codex/20260308T183553_REVIEW_Intent-vector-refinements-composition-vs-unification.md` — review (primary)
- `.ai-workspace/comments/codex/20260308T182037_REVIEW_PLAN-universal-intermediary-scope-and-typing.md` — review